Chess Blunder-Risk Analyzer: D2 Detailed Design

Braden Hayes, Nate Poppe

Header, conventions, and D2 overview

Goal statement. The goal of the Chess Blunder-Risk Analyzer is to develop a machine-learning-based system that estimates the likelihood that a chess player will make a blunder from a given position. The system will analyze completed chess games using factors such as player rating, clock time, game phase, position characteristics, and engine-derived information. It will then present move-level risk scores and broader patterns that help players identify situations in which they are more likely to make mistakes.

Diagram conventions. Every D2 diagram carries the project title, a one-line goal statement, and its own legend.

Figure	What the boxes mean	What the lines mean
Figure 2: Import lifecycle	Rounded boxes are the states of one import. Red boxes are error states. The dark pill is the finished state, and the dot is the start.	Arrows are transitions, labeled with what causes them. Solid gray arrows are the normal path; red dashed arrows are error transitions.
Figure 3: Analyzing one move	Rectangles are steps, diamonds are yes/no decisions, red boxes are error handling, and the dark pill is the end.	Solid gray arrows are the normal path; red dashed arrows are error paths.
Figure 4: D2 data model additions	Each box is a database table, with columns on the left and types on the right. PK is a primary key, FK a foreign key, UK unique, and IX indexed.	Each line is a relationship between two tables. The symbols and green text at each end show how many rows can be on that side: 1 is exactly one, 0..N is zero or more.

Overview

The Chess Blunder-Risk Analyzer lets a player sign in with Lichess, import up to 50 recent games (or upload a PGN file), and see which of their moves carried the highest risk of a blunder and why. Behind the scenes, each game is imported and checked, every one of the player's moves is analyzed by Stockfish, a machine-learning model turns that analysis into a 0 to 100 risk score, and the riskiest moves are explained in plain language.

D1 covered the data model, the import process, and the engine analysis, with an API for starting imports, checking progress, and reading a game's scores.

D2 adds the remaining three components (User Interface, Blunder-Risk Model, and Risk Explanation and Analytics), the glue around them (sign-in, configuration, logging), an edge-case table for every component, a traceability table, and our design rationale. It also adds two diagrams with error paths (Figures 2 and 3) and one new table for risk factors (Figure 4).

Two changes came out of our peer review: stuck imports are now recovered (PR-01), and missing game data now has fallback rules (PR-02).

Known gaps and open questions

- **Training label, training data, and model type** are still being decided with our advisor. D2 defines what the model takes in and gives back, not how it is trained.
- **The exact feature list** will be set by Braden once the model type is chosen. The database stores features in a flexible column so this doesn't block the rest of the system.
- **Analysis speed** has not been measured. The 180-second target in AC-01.1 depends on the engine search depth, which we will tune on the demo machine.
- **Player-level patterns** come from the goal statement, not from a user story. We designed them, but they are the lowest priority (see the reverse check).
- **Hosting** is planned as Docker on a local machine, pending our advisor.
- **Lichess resume behavior** (asking only for games older than a given time) needs to be confirmed against the live API.

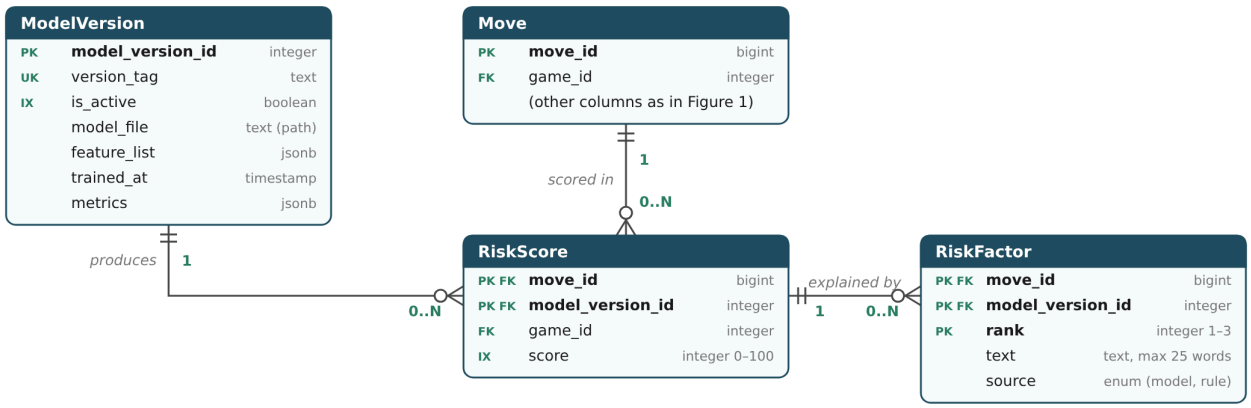
Finished components

Each component below has a data model, its behavior, and its interface. Components finished in D1 (Game Import and Validation, Game and Results Storage, and Chess Feature Extraction) are not repeated.

Chess Blunder-Risk Analyzer: D2 data model additions (Figure 4)

Goal: estimate how likely a player is to blunder from a given position, and show the riskiest moves and why.

New table: RiskFactor. Changed: ModelVersion gains columns. Move is shown for context only.



Boxes are tables, columns on the left and types on the right. Tags: PK primary key, FK foreign key, UK unique, IX indexed. Line ends: 1 exactly one, 0..N zero or more.

User Interface

Owner: Nate Poppe

Data model. No persistent data. The interface keeps page state in the browser while it is open. Everything that needs to last is stored by the backend.

Behavior. The interface has five pages:

1. **Sign in.** Sends the player to Lichess to sign in, then returns them to the Import page.
2. **Import.** The player enters a username, time control, and number of games, or uploads a PGN file. Before confirming, the page shows how many matching games are available. After confirming, it checks the import's progress every second and shows "n of N imported" until the import is done or interrupted. If interrupted, it shows a message and a Resume button. The states the page shows are the ones in Figure 2.
3. **Games.** Lists the player's analyzed games, newest first, each with its three riskiest moves.
4. **Replay.** Shows one game on a board. Selecting a move jumps to that position and shows its risk score, its risk factors, and the engine's best move.
5. **Patterns.** Shows the situations where the player tends to make risky moves, across all their analyzed games.

Interface. The pages call the endpoints listed in the API sections of D1 and of this document. The interface never talks to the database, the engine, or the model directly.

Blunder-Risk Model

Owner: Braden Hayes

Data model. ModelVersion (Figure 4) gains four columns: the path to the saved model file, the list of features the model expects, when it was trained, and its evaluation results. The model file itself is stored on disk, not in the database.

Behavior.

1. When the server starts, it loads the active model version. If the model file is missing, the status page reports the model as down and new imports are turned down.
2. For each batch of moves, the model first checks that the features match the list it was trained on. If they don't match, the whole batch is refused and logged, and those moves stay unscored.
3. Missing values (for example, no clock data from a PGN file) are filled in using the model's own preprocessing rules.
4. The model predicts a blunder probability for each move. The risk score is that probability times 100, rounded.
5. If a single move fails, it gets an empty score and is logged. A bad move never stops the rest of the batch.
6. Releasing a new version: the new model scores the same stored games as the current one, the results are compared, and only then is it marked active. Old scores are kept so the comparison can be repeated.

Training itself happens offline and is not part of the running system. How it is done depends on our advisor meeting.

Interface.

Method	Takes	Gives back	Fails when
`load_active()`	Nothing	The active model and its version	The model file is missing or can't be read
`predict(rows)`	A list of moves, each with its features	For each move: its risk score (0 to 100, or empty) and the model version used	The features don't match what the model expects (whole batch refused)
`contributions(row)`	One move's features	How much each feature pushed the score up or down	The model type doesn't support it (the caller falls back to rule-based factors)

Risk Explanation and Analytics

Owner: Braden Hayes

Data model. One new table, RiskFactor (Figure 4). Each scored move can have up to three factors, each a short sentence and a note of whether it came from the model or from a rule. Player patterns are calculated when requested and are not stored.

Behavior: explaining a move.

1. Ask the model which features pushed this move's score up the most.
2. Turn each of those features into a plain-language sentence using a fixed phrase table (for example, low clock time becomes "You had less than 10% of your time left").
3. If the model can't give reasons, use rule-based factors instead, based on the raw features (low clock, many legal moves, a large gap between the best and second-best move).
4. Keep the top two or three factors. Drop any factor over 25 words or containing an internal feature name.
5. Save the factors with the score.

Behavior: finding patterns.

1. Collect all of the player's scored moves.
2. Group them by game phase, time left on the clock, and position type.
3. Skip any group with fewer than 10 moves, since it is too small to mean anything.
4. Return the groups where the player's average risk is highest, with a short description of each.

Interface.

Method or endpoint	Takes	Gives back	Fails when
`explain(scored_moves)`	Scored moves with their features	Up to three factors per move	Never fails as a whole; a move with no usable factors gets none and is logged
`GET /v1/patterns`	The signed-in player	Up to five risky situations, each with a description, average risk, and number of moves	The player has no analyzed games (returns an empty list with a message)

Glue components

Authentication (owner: Nate Poppe) - **Data model:** uses the existing User table. Sessions are kept in a signed browser cookie, so there is no session table. - **Behavior:** the player is sent to Lichess to sign in. When they come back, the server finds or creates their User row and sets the cookie. Every other endpoint checks the cookie first.

Configuration (owner: Nate Poppe) - **Data model:** no persistent data. Settings come from an environment file: database address, Stockfish path, search depth, number of engines, model file path, and Lichess sign-in keys. - **Behavior:** settings are read once when the server starts. A missing required setting stops the server from starting and names the setting. The number of engines is capped at one less than the number of CPU cores.

Logging (owner: Nate Poppe) - **Data model:** no database. Log lines go to a file and to the console, and each includes the import, game, and move it is about. - **Behavior:** every failure in Figures 2 and 3 writes a log line. The log file rotates daily and keeps 14 days. Session cookies and sign-in keys are never written to the log.

New and changed endpoints

Endpoint	What it does	Sends	Returns	Fails when
`GET /v1/lichess/players/{username}/summary`	Shows how many matching games are available before the player confirms	Username and time control	The number of matching games	The username doesn't exist, or Lichess can't be reached
`GET /v1/games`	Lists the player's analyzed games	How many to show (optional, up to 50)	Each game with its date, opponent, result, and three riskiest moves	The player isn't signed in

Endpoint	What it does	Sends	Returns	Fails when
`GET /v1/games/{id}/moves` (changed)	Now also includes risk factors	Same as D1	Same as D1, plus up to three factors for each scored move	Same as D1
`GET /v1/patterns`	Shows the player's risky situations	Nothing	See Risk Explanation and Analytics	The player isn't signed in
`GET /v1/auth/login`	Starts sign-in with Lichess	Nothing	Sends the player to Lichess	Lichess sign-in is down
`GET /v1/auth/callback`	Finishes sign-in	What Lichess sends back	Signs the player in and returns them to the Import page	The player cancelled, or the sign-in check doesn't match
`POST /v1/auth/logout`	Signs the player out	Nothing	Confirmation	Never
`GET /v1/status`	Reports whether the engine and model are running	Nothing	Engine and model status	Never

Adding risk factors to the moves endpoint is an added field, so under our D1 versioning rule it is not a breaking change.

Changes from peer review

Stuck imports (PR-01). If the server stops in the middle of an import, that import would stay marked as running forever, and the player could never start another one. Now, when the server starts and once a minute while it runs, any import that is marked running but hasn't saved a game for a set time (planned at 5 minutes) is changed to interrupted. The player can then resume it as normal (Figure 2).

Missing game data (PR-02). PGN files from different sites don't all include the same information. Feature Extraction now follows these fallback rules instead of failing:

Missing data	Fallback
Clock times	Clock features are left empty and a "clock available" flag is set to no. The model fills them in using its preprocessing rules.

Missing data	Fallback
Player's rating	The typical rating for that time control is used, and a "rating available" flag is set to no.
Opponent's rating	Rating difference is set to zero, and the same flag is set.
Time control	Treated as unknown; time-pressure features are left empty.
Result or how the game ended	If the game can't be confirmed as finished, it is skipped, since unfinished games are never analyzed.
Readable move list	The game is skipped and counted as skipped.

The flags let the model tell a real value from a filled-in one. Risk explanations never mention a filled-in value, so a player is never told "you were low on time" when the clock wasn't recorded.

Revisions to D0 interfaces

Interface	Change	Why
I1 (UI to Import)	Split into separate endpoints for Lichess imports, PGN uploads, progress, and resume	One interface was carrying four different requests with different inputs.
I4 (Storage to Feature Extraction)	Now passes only the player's moves with their positions and clock times	Passing the whole stored game leaked storage details the engine step doesn't need.
I6 and I7 (Features to Model to Explanation)	Defined as `predict` and `contributions` (see the Model's interface)	The D0 version mixed feature values, scores, and explanation data in one message.
I8 (Explanation to Storage)	Now writes a RiskScore row and its RiskFactor rows together in one step	A score is never saved without its factors, or the other way around.
I9 (Storage to UI)	Now the read endpoints: games list, game moves, and patterns	Clearer than one general "results" interface.

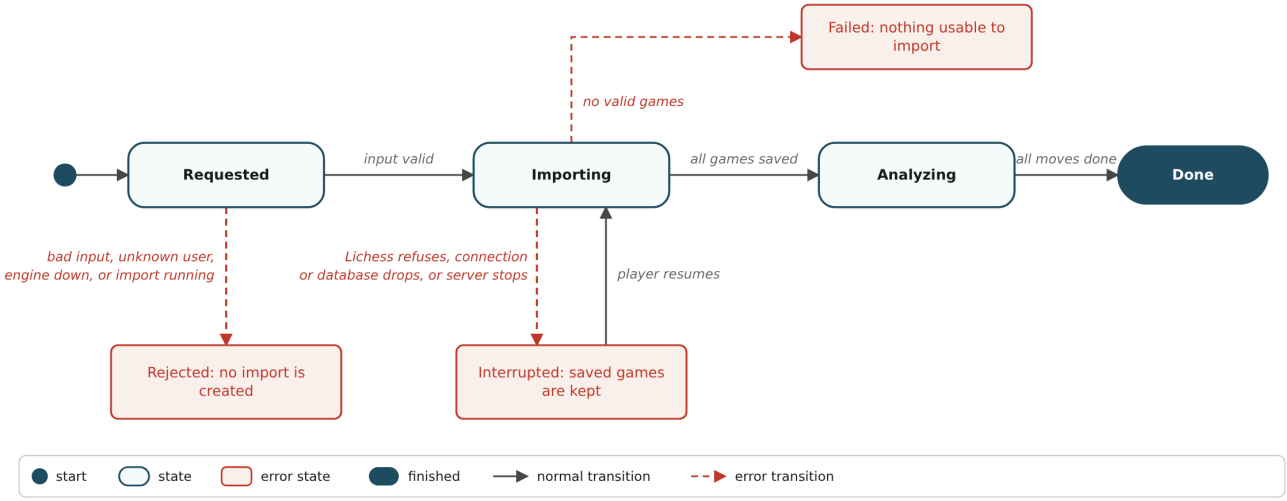
Edge cases and error handling

Figures 2 and 3 show the error paths for the two riskiest flows: importing games and analyzing each move.

Chess Blunder-Risk Analyzer: Import lifecycle (Figure 2)

Goal: estimate how likely a player is to blunder from a given position, and show the riskiest moves and why.

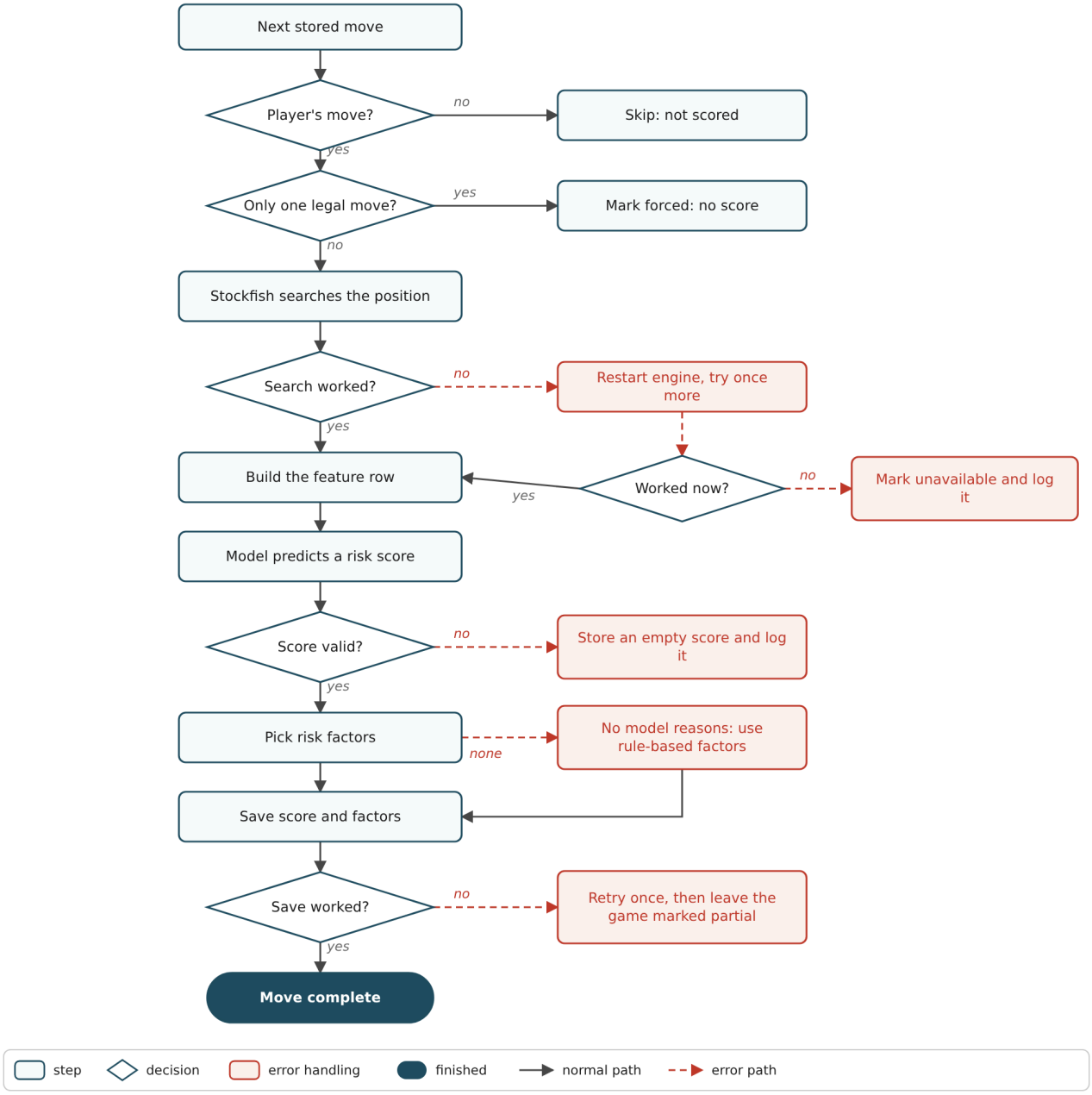
State machine for one ImportJob. Error states are red; the player can resume an interrupted import.



Chess Blunder-Risk Analyzer: Analyzing one move (Figure 3)

Goal: estimate how likely a player is to blunder from a given position, and show the riskiest moves and why.

Runs for every stored move after import. Red dashed paths are failures and how each is handled.



ID	Component	Category	Edge case	Decision	Handler	Recoverable
EC-01	Import	Boundary values	Number of games is 0 or more than 50	Surface error: the form asks for 1 to 50	Import endpoint	Yes, the player fixes it

ID	Component	Category	Edge case	Decision	Handler	Recoverable
EC-02	Import	Timing and concurrency	Player starts a second import, or presses resume twice	Surface error: second request turned down	Import service	Yes
EC-03	Import	Bad or missing input	One game in a PGN file is broken	Degrade: skip that game, report how many were skipped	Import validator	Yes
EC-04	Import	External dependencies	Lichess refuses requests or the connection drops	Surface error: import marked interrupted, saved games kept	Import service	Yes, the player resumes
EC-05	Import	Bad or missing input	A game is still being played	Degrade: skip it, never analyze it	Import validator	Yes
EC-06	Storage	Boundary values	Player has no games yet	Degrade: empty list with a message	Games endpoint	Yes
EC-07	Storage	Timing and concurrency	The same game is saved twice during a resume	Degrade: unique index ignores the duplicate	Database	Yes
EC-08	Storage	Bad or missing input	A score points to a model version that doesn't exist	Surface error: write refused and logged	Database	No, it is a bug to fix
EC-09	Storage	External dependencies	Database can't be reached	Surface error: import marked interrupted, page shows the problem	Storage layer	Yes, once the database is back
EC-10	Feature Extraction	Boundary values	Move had only one legal option	Degrade: mark forced, no score	Analysis worker	Yes

ID	Component	Category	Edge case	Decision	Handler	Recoverable
EC-11	Feature Extraction	Timing and concurrency	Server restarts during analysis	Retry: unanalyzed moves are queued again at startup	Analysis worker	Yes
EC-12	Feature Extraction	Bad or missing input	PGN game has no clock times	Fall back: clock features left empty and flagged; model fills them in; no clock-based explanations (PR-02)	Feature builder	Yes
EC-13	Feature Extraction	External dependencies	Stockfish crashes or freezes	Retry once, then mark unavailable and log	Analysis worker	Yes for the next move
EC-14	Model	Boundary values	Predicted probability is exactly 0 or 1	Degrade: score kept within 0 to 100	Model	Yes
EC-15	Model	Timing and concurrency	Active model changes during an import	Fall back: the import keeps the version it started with	Model	Yes
EC-16	Model	Bad or missing input	Features don't match what the model expects	Surface error: batch refused, moves left unscored, logged	Model	Yes, after a fix and rescore
EC-17	Model	External dependencies	Model file missing at startup	Surface error: status page shows model down, imports turned down	Configuration check	Yes, once the file is restored
EC-18	Explanation	Boundary values	Fewer than two usable factors for a move	Fall back: rule-based factors	Explainer	Yes

ID	Component	Category	Edge case	Decision	Handler	Recoverable
EC-19	Explanation	Timing and concurrency	Patterns requested while an import is still running	Degrade: use finished games only, labeled as partial	Patterns endpoint	Yes
EC-20	Explanation	Bad or missing input	A factor is over 25 words or contains a feature name	Degrade: drop that factor	Explainer	Yes
EC-21	Explanation	External dependencies	Model can't say which features mattered	Fall back: rule-based factors	Explainer	Yes
EC-22	User Interface	Boundary values	A game has fewer than three scored moves	Degrade: show what exists	Games page	Yes
EC-23	User Interface	Timing and concurrency	Player leaves the page during an import	Degrade: import keeps running; progress shows when they return	Import page	Yes
EC-24	User Interface	Bad or missing input	Username has characters Lichess doesn't allow	Surface error: form blocks it	Import page	Yes
EC-25	User Interface	External dependencies	Backend can't be reached	Retry: "can't reach the server" message, retried every few seconds	All pages	Yes
EC-26	Authentication	Boundary values	Session has expired	Surface error: send to sign-in, then back to the same page	Auth check	Yes

ID	Component	Category	Edge case	Decision	Handler	Recoverable
EC-27	Authentication	Timing and concurrency	Player signs out in another tab during an import	Degrade: import keeps running; results visible after signing in again	Auth check	Yes
EC-28	Authentication	Bad or missing input	Cookie has been tampered with, or the sign-in check doesn't match	Surface error: turned down, sign in again	Auth check	Yes
EC-29	Authentication	External dependencies	Lichess sign-in is down	Surface error: "try again later" message	Auth endpoints	Yes
EC-30	Configuration	Boundary values	Number of engines set higher than the CPU allows	Degrade: capped at cores minus one	Configuration loader	Yes
EC-31	Configuration	Timing and concurrency	A setting is changed while the server runs	Degrade: ignored until restart	Configuration loader	Yes
EC-32	Configuration	Bad or missing input	A required setting is missing	Surface error: server won't start and names the setting	Configuration loader	Yes
EC-33	Configuration	External dependencies	Stockfish path is wrong	Surface error: status page shows engine down	Configuration check	Yes
EC-34	Logging	Boundary values	Log file grows large	Degrade: rotate daily, keep 14 days	Logger	Yes
EC-35	Logging	Timing and concurrency	Several engines log at the same moment	Degrade: each line is written whole	Logger	Yes

ID	Component	Category	Edge case	Decision	Handler	Recoverable
EC-36	Logging	Bad or missing input	A log line would include a cookie or sign-in key	Degrade: those fields are removed before writing	Logger	Yes
EC-37	Logging	External dependencies	Disk is full	Fall back: log to the console only; the app keeps running	Logger	Yes
EC-38	Import	Timing and concurrency	Server stops during an import, leaving it stuck as running	Recover: marked interrupted after a set time with no progress, so the player can resume (PR-01)	Import service	Yes
EC-39	Feature Extraction	Bad or missing input	PGN game is missing ratings or time control	Fall back: use the rules in "Changes from peer review" and flag the filled-in values (PR-02)	Feature builder	Yes

Requirements traceability

Functional requirements come from our User Stories document. Non-functional requirements marked "derived" come from D0 or from what a user story implies.

ID	Requirement	Priority	Design elements	How	Edge cases and figures
US-01	Player finds their riskiest moves and why	Must	`GET /v1/games`, `GET /v1/games/{id}/moves`, RiskScore index, RiskFactor table, `explain`	Games list shows each game's three riskiest moves; replay shows each move's factors	EC-18, EC-22; Figures 3 and 4

ID	Requirement	Priority	Design elements	How	Edge cases and figures
US-02	Every score traces to the model version that made it	Must	ModelVersion, RiskScore keyed by move and version, `model_version` option on the moves endpoint, Model release steps	Old scores are kept, so two versions can be compared on the same games	EC-08, EC-15; Figure 4
US-03	Games still being played are never analyzed	Must	A1 validation, Game holds finished games only	Unfinished games are skipped during import and never reach the engine	EC-05; Figure 2
UC-01	Main flow: sign in, import, see progress, see riskiest moves, open replay	Must	Sign-in endpoints, summary endpoint, import endpoints, Import and Replay pages	Each step of the use case maps to one page and one endpoint	EC-01, EC-02; Figure 2
UC-01 A1	Upload a PGN file instead of a username	Must	`POST /v1/imports/pgn`, A1 validator	Same validation and saving as a Lichess import; missing fields follow fallback rules	EC-03, EC-12, EC-39
UC-01 E1	Import is interrupted and can be resumed	Must	Interrupted state, resume endpoint, Game index on played_at	Saved games are kept; resume only asks for older games; stuck imports are recovered	EC-04, EC-09, EC-38; Figure 2
UC-01 pre 1	Player is signed in	Must	Authentication	Every endpoint checks the session first	EC-26 to EC-29
UC-01 pre 3	Status page shows engine and model are running	Must	`GET /v1/status`, startup checks	Import page checks status before allowing an import	EC-17, EC-33

ID	Requirement	Priority	Design elements	How	Edge cases and figures
UC-01 post	Games and scores stored per player with no duplicates	Must	Unique index on user and game, per-game save	Duplicates are ignored by the database	EC-07
AC-01.1	20 games and their top three moves within 180 seconds	Must	A2 (parallel engines, fixed search), RiskScore index	Engines run side by side; search depth will be tuned to meet the target	EC-13, EC-22; Figure 3. **Gap:** not yet measured.
AC-01.2	Replay shows the exact position and at least two plain factors, each 25 words or fewer	Must	`fen_before` on Move, RiskFactor, phrase table, rule-based fallback	Board jumps to the stored position; factors are checked for length and wording	EC-18, EC-20, EC-21; Figure 4
AC-01.3	Interrupted import keeps saved games, shows a message within 2 seconds, re-downloads nothing	Must	A1, per-game save, progress checked every second, resume endpoint	Message appears on the next progress check	EC-04; Figure 2
NFR-01 (derived from US-02)	Rerunning a game gives the same result	Must	Fixed search depth in A2, stored engine results	Same input always gives the same features and score	EC-11, EC-13
NFR-02 (derived)	One player's data is never shown to another	Must	Game owned by one User, every read filtered by the signed-in player	Another player's import or game looks the same as one that doesn't exist	EC-28
NFR-03 (derived from D0)	Runs on a standard computer	Should	Configuration (engine count capped), Docker on a local machine	No special hardware needed	EC-30

ID	Requirement	Priority	Design elements	How	Edge cases and figures
GOAL-01 (from the goal statement)	Show broader patterns across a player's games	Could	`GET /v1/patterns`, pattern steps	Groups scored moves by situation	EC-19. **Gap:** no user story; see reverse check.

Reverse check

Component or element	Requirements behind it	Note
User Interface	US-01, UC-01, AC-01.2, AC-01.3	Covered
Game Import and Validation	US-03, UC-01, UC-01 A1, UC-01 E1, AC-01.3	Covered
Game and Results Storage	US-02, UC-01 post, NFR-02	Covered
Chess Feature Extraction	AC-01.1, NFR-01	Covered
Blunder-Risk Model	US-01, US-02	Covered
Risk Explanation (factors)	US-01, AC-01.2	Covered
Risk Analytics (patterns)	GOAL-01 only	**Possible scope creep.** No user story asks for it. We keep it as the lowest priority and build it last.
Authentication	UC-01 pre 1, NFR-02	Covered
Configuration and status page	UC-01 pre 3, NFR-03	Covered
Logging	None directly	Supports the error handling D0 requires for I5, I7, and I8. Kept because every error path in Figures 2 and 3 depends on it.

Design rationale

These cover the significant decisions in D1 and D2. Decisions about architecture, using Stockfish, the two import methods, and model output are in the D0 decision log and are not repeated. Decisions changed by our peer review are marked.

1. **Database.** We chose a relational database (PostgreSQL) over a document store because our key rules (no duplicate games, every score tied to a real model version) can be enforced by the database itself. We accepted the trade-off of writing migrations

whenever a table changes because the rules it guarantees would otherwise be hand-written and easy to break.

2. **Saving games.** We chose to save each game in its own step over saving the whole import at once because an interrupted import keeps every game saved so far. We accepted the trade-off of the player briefly seeing a partly imported list because that is exactly what AC-01.3 asks for.
3. **Engine search.** We chose a fixed amount of search work per position over a fixed amount of time because results stay the same from run to run, which is needed to compare model versions. We accepted the trade-off of some positions taking longer than others because the total stays predictable and we can tune it.
4. **Keeping engine results.** We chose to store engine results in their own table over recomputing them because engine analysis is the slowest step. We accepted the trade-off of extra storage because a new model can then rescore old games without running the engine again.
5. **Keeping old scores.** We chose to keep scores from every model version over overwriting them because it lets us compare a new version with the old one on the same games. We accepted the trade-off of the scores table growing with each release because releases are rare and each is small.
6. **Progress updates.** We chose to have the page check progress every second over a live connection (WebSockets) because it is simpler and works through any network. We accepted the trade-off of extra small requests because they are cheap and still meet the 2-second message requirement.
7. **Sign-in.** We chose Lichess sign-in over our own accounts because we don't store passwords and it proves the player owns the Lichess account. We accepted the trade-off of PGN-only players needing a Lichess account because almost all of our users import from Lichess.
8. **Explanations.** We chose fixed phrases with a rule-based fallback over generated text because every factor stays short, readable, and free of internal names. We accepted the trade-off of less varied wording because AC-01.2 limits each factor to 25 words with no feature names.
9. **Background work.** We chose a simple queue inside the backend over a separate queue service (Celery with Redis) because the system runs on one machine. We accepted the trade-off of not being able to spread work across machines because the D0 decision against microservices already rules that out for now. **(Changed by peer review, PR-01:** because work lives inside the server, a crash could leave an import stuck as running, so stuck imports are now marked interrupted after a set time with no progress.**)**
10. **Missing game data (changed by peer review, PR-02).** We chose to fill in missing values with flagged fallbacks over rejecting incomplete games because PGN files from other sites often leave out clocks or ratings, and rejecting them would make PGN upload far less useful. We accepted the trade-off of less accurate scores for those

games because the flags let the model account for filled-in values, and explanations never rely on them.

Peer design review notes

These notes are also committed separately as `Peer_Review_Notes.md`.

Feedback we received

ID	Criterion	Issue (and team that raised it)	Action	Owner	Priority	Due
PR-01	Completeness / Risk	The API turns down a new import while one is running. If the application crashes mid-import, that import stays marked as running and blocks every future import. (Team 22)	Add a timeout that changes a stuck import to interrupted so it can be resumed. Added to D2: "Changes from peer review", EC-38, Figure 2, rationale 9.	Nate Poppe	Worth considering	Week 9
PR-02	Completeness	A user could upload a PGN file missing data such as clock times, since different platforms export different fields. (Team 22)	Create fallback rules for missing values during feature extraction. Added to D2: "Changes from peer review", EC-12, EC-39, rationale 10.	Braden Hayes	Must-fix	Week 9

The must-fix item (PR-02) was applied to D2 before submission. PR-01 was also small enough to add now.

Summary

Criterion with the most feedback: Completeness, which came up in both issues. One of them (PR-01) was also a risk concern.

Top two issues we raised while reviewing other teams:

We raised four issues across two teams. The top two, both rated must-fix:

1. **Team 22: Nate Heath, Elliot Warner, and William Thomas (Completeness):** when users switch applications with the dial, it isn't defined whether the previous application keeps running or stops, or how its state is kept. We suggested defining the application lifecycle, including state and hardware resources.
2. **Team 49: Cameron Pocisk and Jake Skaff (Feasibility and Risk):** the Drawing Bridge relies on handing colors to drawing applications, but those apps may not share a way to set the active color. We suggested choosing the first supported apps and confirming each one can have its color set by a program.